**1. Path Length** **05A1**

将路径的长度定义为边的总数而非顶点的总数，相对而言有何便利之处?

**2. Depth & Height** **05A1**

a) 试证明：树中任何节点$v$都满足：$\text{depth}(v) + \text{height}(v) \leq \text{height}(T)$;

b) 何时取等号?

**3. Tree Representation** **05A2**

分别使用父节点表示法、孩子节点表示法、父节点-孩子节点表示法：

a) 如何判定一对节点是否为祖先-后代关系？为此需要花费多少时间?

b) 如何支持节点的接入?

c) 如何支持子树的删除?

d) 如何找出任意一对节点的最低公共祖先（lowest common ancestor）?

**4. Binary Tree** **05B1**

将二叉树中叶节点的总数记作$n_0$，单分支节点的总数记作$n_1$，双分支节点的总数记作$n_2$，边数记作$e$。

a) 试证明，任何一棵二叉树都满足：$n_0 = n_2 + 1$;

b) 试证明，任何一棵二叉树都满足：$e = n_0 + n_1 + n_2 - 1 = n_1 + 2 \cdot n_2$;

（推论：$2 \cdot n_2 = e = 2 \cdot (n_0 - 1)$当且仅当是真二叉树。）

**5. Binary Tree** **05B1**

我们在课上已看到，在有根、有序的前提下，任何一棵多叉树经过变换都唯一地对应于一棵二叉树。

a) 反过来，任何一棵二叉树是否也会对应于某棵多叉树?

b) 没有父亲的树根，固然无所谓是左孩子或右孩子，但是我们为什么还是倾向于将树根视作右孩子?

**6. Binary Forest** **05B1**

若干棵有根有序的多叉树，可以统称作森林；若其中的多叉树之间也有次序，则称作有序森林。

试证明：在有序森林与二叉树之间，存在一一对应的关系。

**7. Proper Binary Tree** **05B1**

试设计一个对应关系，使得由$n_2 + 1$个叶节点、$n_1$个单分支节点和$n_2$个双分支节点构成的每棵二叉树，都对应于一棵由$2 \cdot n_2 + n_1 + 2$个叶节点、$0$个单分支节点和$2 \cdot n_2 + n_1 + 1$个双分支节点构成的真二叉树。

**8. updateHeightAbove()** **05B2**

为确保所有祖先的高度都能更新，示例代码中实现的算法会一直逐层上溯到根。

a) 在某节点作为叶子插入之后，如果某个祖先的高度没有变化，那么更高祖先的高度仍有可能会有变化吗?
若可能，试举一例；否则，试给出证明;

b) 在某个叶节点被删除之后呢?

c) 在某棵子树接入（attach()）之后呢?

d) 在某棵子树分离（secede()）之后呢?

e) 基于以上结论，可如何优化updateHeightAbove()算法?

f) 如何评估你的优化效果?

**9. Energetic vs. Lazy** **05B2**

二叉树的height、size等信息，可以有两种记录方式。在我们的示例代码中，前者采用“勤奋策略”，每个节点的高度一旦有变化，便立即更新；后者则采用“懒惰策略”，直到需要时才调用size()递归地统计。

a) 试颠倒上述策略，修改代码并完成测试;

b) 两种策略各有什么优点、缺点，分别适用于哪些应用场合?

**10. Head/tail recursion** **05C1**

考查二叉树的递归式中序遍历算法。

a) 对于什么样的二叉树，遍历的过程会相当于尾递归?

b) 对于什么样的二叉树，遍历的过程会相当于头递归（递归发生在函数中最靠前的位置，仅次于递归基）?

**11. Tail recursion** **05C1**

考查二叉树的递归式先序遍历算法。

a) 试验证：在二叉树退化成单链（所有节点至多只有一个孩子）时，遍历的过程相当于尾递归;

b) 此时，时间、空间复杂度可以如何优化?

c) 如果BinNode没有设置parent引用，你的优化是否依然可行?

**12. BinNode::size()** **05C1**

本节介绍的二叉树规模统计算法，除了先序遍历，也可以改用中序、后序或层次遍历;

a) 试通过定义一个函数对象visit()，直接调用BinNode::traverse(x, visit())以实现同样的功能;

b) 本节所举的其它应用实例，是否也可以这样改写?

**13. clone()** **05C1**

本节介绍的二叉树深复制算法，采用了先序遍历的策略。是否可以改用中序或后序策略?

如果可以，具体如何实现？否则，为什么不可以?

**14. printBinTree()** **05C1**

本节讲义仅介绍了printBinTree()算法的基本原理及梗概。试从实例代码包中找到该算法的详细代码，阅读并了解其具体而完整的实现方式，尤其是如何展示出各节点的深度，并绘制出方向正确的联边。

**15. clean()** **05C1**

本节介绍的二叉树清空算法，采用了后序遍历的策略。是否可以改用先序或中序策略?

如果可以，具体如何实现？否则，为什么不可以?

**16. RPN ~ Postorder** **05C1**

在本节中我们看到，只含一元、二元运算符的合法表达式总是可以转换为二叉树，并进而通过后序遍历得到对应的RPN。那么反过来，可否由PRN得出对应的二叉树呢？若可以，方法如何？若不可以，原因何在?

**17. Iterative Preorder Traversal** **05C2**

讲义中介绍了先序遍历的一种迭代算法，其思路是沿着左侧通路逐层地分解二叉树。

a) 试对称地沿着右侧通路来做分解，实现先序遍历的另一个迭代算法;

b) 相对于讲义中的算法，新的这个算法有何优点?

c) 有何缺点?

**18. Correctness Of Iterative Preorder Traversal** **05C2**

a) 试通过对二叉树的规模做数学归纳，证明本节给出的迭代式先序遍历算法是正确的。

提示:考查藤上所有节点均被访问之后的时刻——此时,原树接下来的遍历过程,可分解为对多棵右子树的遍历;这些子树的规模均有所削减，且彼此独立。

b) 对藤的长度做数学归纳，是否同样可以证明？如果不行，试说明理由；否则，给出你的证明。

**19. Empty right subtrees** **05C2**

在迭代式先序遍历preorder1()算法中，内层for循环的作用是沿藤下行，并将各右子树的根节点推出栈中。然而我们不难发现，其中有些右子树可能是空的。 这类空的子树随后也会在主算法中出栈，虽然不致错误，却毕竟是多余的“过门”。

a) 如何避免此类没有实效的入栈、出栈?

b) 在你改进之后，算法的总体效率是否会有（当然，只是常系数意义上的）提高?

**20. Storage cost of iterative preorder traversal** **05C2**

考查本节所介绍迭代式先序遍历算法的空间复杂度。

a) 试证明：该复杂度不会超过$T$的高度；

b) 试证明：该复杂度更精确的估计是$\mathcal{O}(\max\{\ la(v)\ |\ v \in T\ \})$，其中$la(v)$为节点$v$所有左祖先的总数；

c) 当然，后一上界不致超过前者，但二者的差距最大可能多大？试构造出如此极端的$T$；

d) 试构造另一极端的$T$来说明：两个上界也可能相等；

e) 两个上界之比值的数学期望是多大？为什么?

**21. Correctness of iterative inorder traversal** **05C3**

我们在课上通过对二叉树的规模做数学归纳，证明本节给出的迭代式中序遍历算法是正确的。

如果改为对藤的长度做数学归纳，是否同样可以证明?

如果可行，试给出你的证明；否则，试说明理由。

**22. Last inorder node** **05C4**

无论何种遍历，每一棵二叉树都有且仅有一个最终被访问的节点。当然，对于这个节点而言的succ()应该返回**nullptr**。

阅读本节针对中序遍历所实现的succ()算法，理解这一功能是如何落实的。

**23. Parent in postOrder()** **05C5**

本节的迭代式后序遍历算法需要借助parent信息，来区分“返回父节点”或“展开右子树并遍历之”这两种情况。

a) 如果BinNode结构没有记录parent信息，该算法可否在经过适当调整之后依然可行?

b) 如果依然可行，试说明应如何调整；否则，试解释为何不能。

**24. Correctness of iterative postorder traversal** **0C5**

仿照课上针对中序遍历的方法，证明本节迭代式后序遍历算法是正确的。

**25. Amortization** **05C[2|3|5]**

任选一种均摊分析的方法以证明，迭代式先序、中序、后序遍历算法的时间复杂度均为$\mathcal{O}(n)$。

**26. Level-order traversal** **05C6**

试证明层次遍历的如下性质，并由此确立算法的正确性及复杂度：

a) 每次迭代中入队的节点（若存在），都是出队节点的孩子；

b) 辅助队列中的各节点，在任何时刻都按深度单调排列，而且深度相差不超过**1**层；

c) 所有节点迟早都会入队，而且更高/低的节点，更早/晚入队；更左/右的节点，更早/晚入队；

d) 每次迭代中尽管入队节点数目（从0至2）不定，但总是恰有一个节点出队并接受访问；

e) 每个节点入、出队恰好各一次，故知整体只需$\mathcal{O}(n)$时间。

**27. Storage cost of level-order traversal** **05C6**

a) 层次遍历的空间成本主要消耗于辅助队列，那么这一成本与二叉树的结构有何关系?

b) 在规模同为$n$的所有二叉树中，哪一棵的层次遍历需要使用最多的空间?

**28. Complete Binary Tree** **05C6**

a) 本节指出，完全二叉树可以向量的形式，紧凑而高效地物理实现。试动手完成这一任务。

b) 这种实现方式能否充分发挥系统缓存的作用？也就是说，连续访问的节点是否在物理上通常会彼此临近?

**29. Rebuilding a proper binary tree** **05C7**

本节指出，仅凭其先序与后序遍历序列，依然可以重建一棵真二叉树，试编程实现这一算法。

**30. Rebuild by augmented sequences** **05C7**

a) 试证明：由先序增强序列，可以构造出对应的二叉树；

b) 试编程实现对应的算法；

c) 中序增强序列呢？

d) 后序增强序列呢？

**31. More rebuildings** **05C7**

各种遍历序列的其它组合，是否可以完成对原二叉树的重建？试逐一考查验证。

**32. Greedy Huffman** **05D1**

a) 试证明Huffman贪心策略的正确性：

任何一对子树交换位置，只要能使频率高/低者在树中的位置更高/低，编码成本便会下降。

b) 下降的数值取决于哪些因素？具体等于多少？

**33. Optimality of Huffman tree** **05D2**

试按照讲义中的思路证明：尽管最优编码树未必唯一，但Huffman算法所构造出来的必属其一。

**34. Implementing Huffman algorithm** **05D3**

试阅读示例代码中Huffman算法的部分，重点厘清以下方面：

a) 算法的主体框架是在何处、如何描述的？

b) 目前是如何实现Huffman森林的？

c) 算法框架的描述，如何能够独立于具体所选用的数据结构？

d) 日后实现Huffman森林的其他更高级的数据结构，是如何与此框架自然接驳的？

**35. Huffman tree using stack and queue** **05D3**

本节介绍了如何借助一个栈和一个队列，简明地实现复杂度为$\mathcal{O}(n \log n)$的Huffman算法。

a) 试确认，在该算法的整个过程中，栈与队列都是单调的；

b) 试确认，栈也可以换成队列（也就是总共使用两个队列）。

**36. Lower bound of average complexity** **05E1**

讲义中我们已借助ADT模型证明：基于比较的任何排序算法，在最坏情况下都需要$\Omega(n \log n)$时间。

试证明：其平均时间复杂度也不会低于$\Omega(n \log n)$。

**37. Lower bound of Huffman encoding** **05E2**

a) 试通过归约证明：最优编码树的任何构造算法，时间复杂度的下界都是$\Omega(n \log n)$；

b) 试证明：**Huffman**编码树的任何构造算法，时间复杂度的下界都是$\Omega(n \log n)$u。